

Lecture 26

Tri-state Logic and Buses

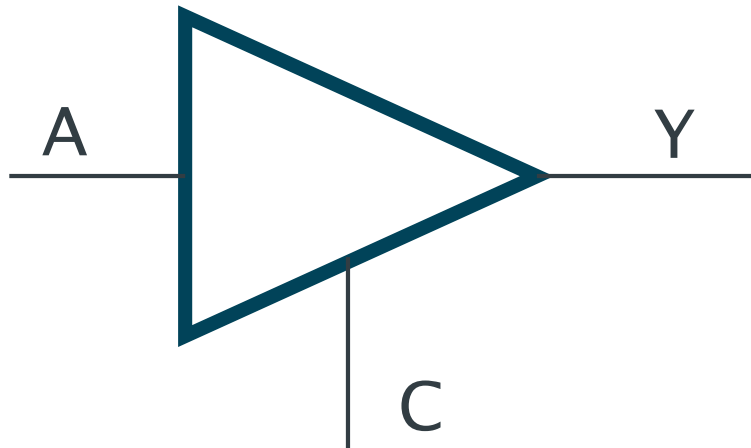
Contents

- Tri-state Logic
 - The tri-state buffer
 - Tri-states and multiplexers
 - Four state logic
- Buses
 - Definition; what are they, and why are they needed?
 - Computer architecture
 - Bus contention
 - Avoiding bus contention
 - Bidirectional buses using tri-states
- An example

26.1

Tri-State Buffer

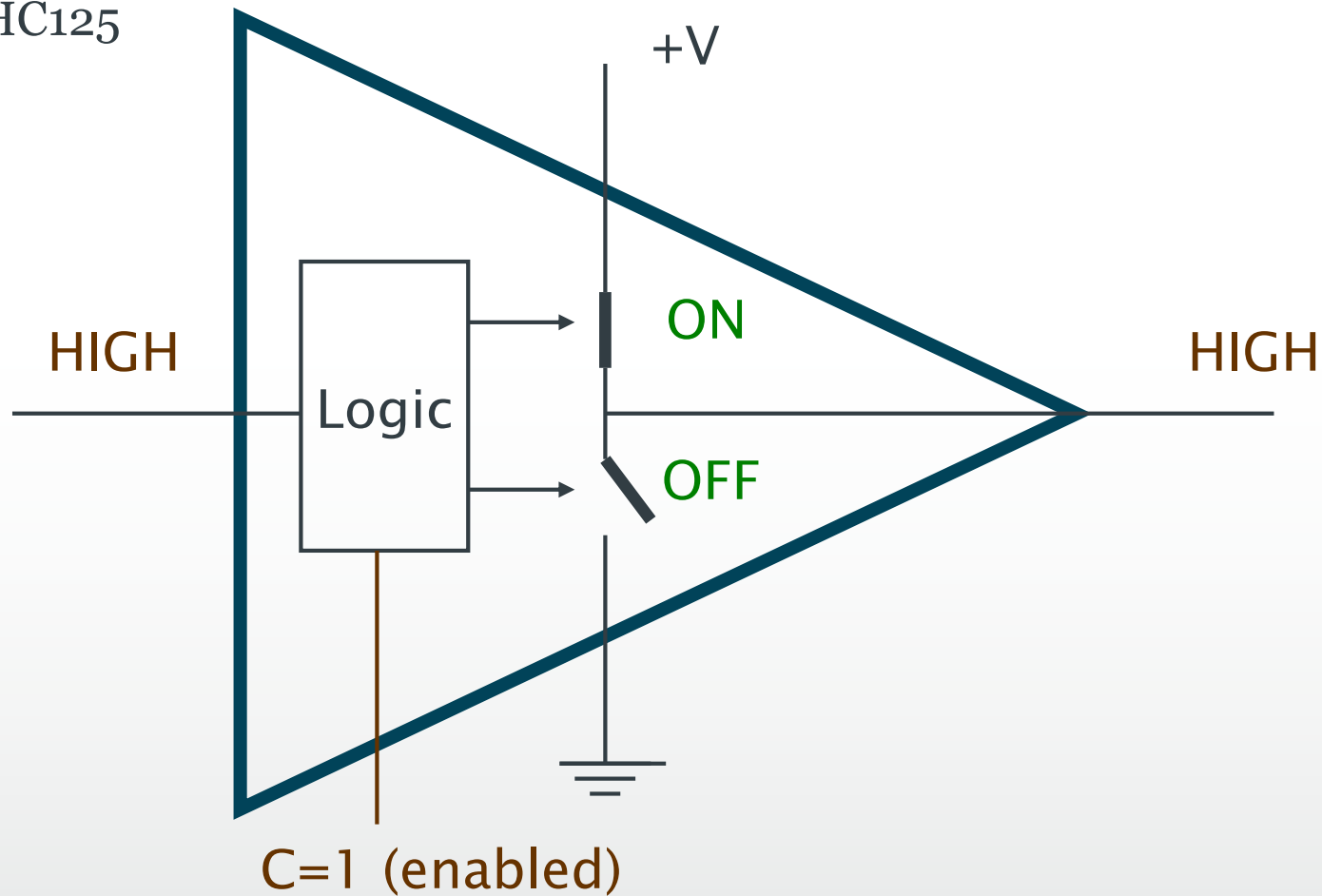
The Tri-State Buffer



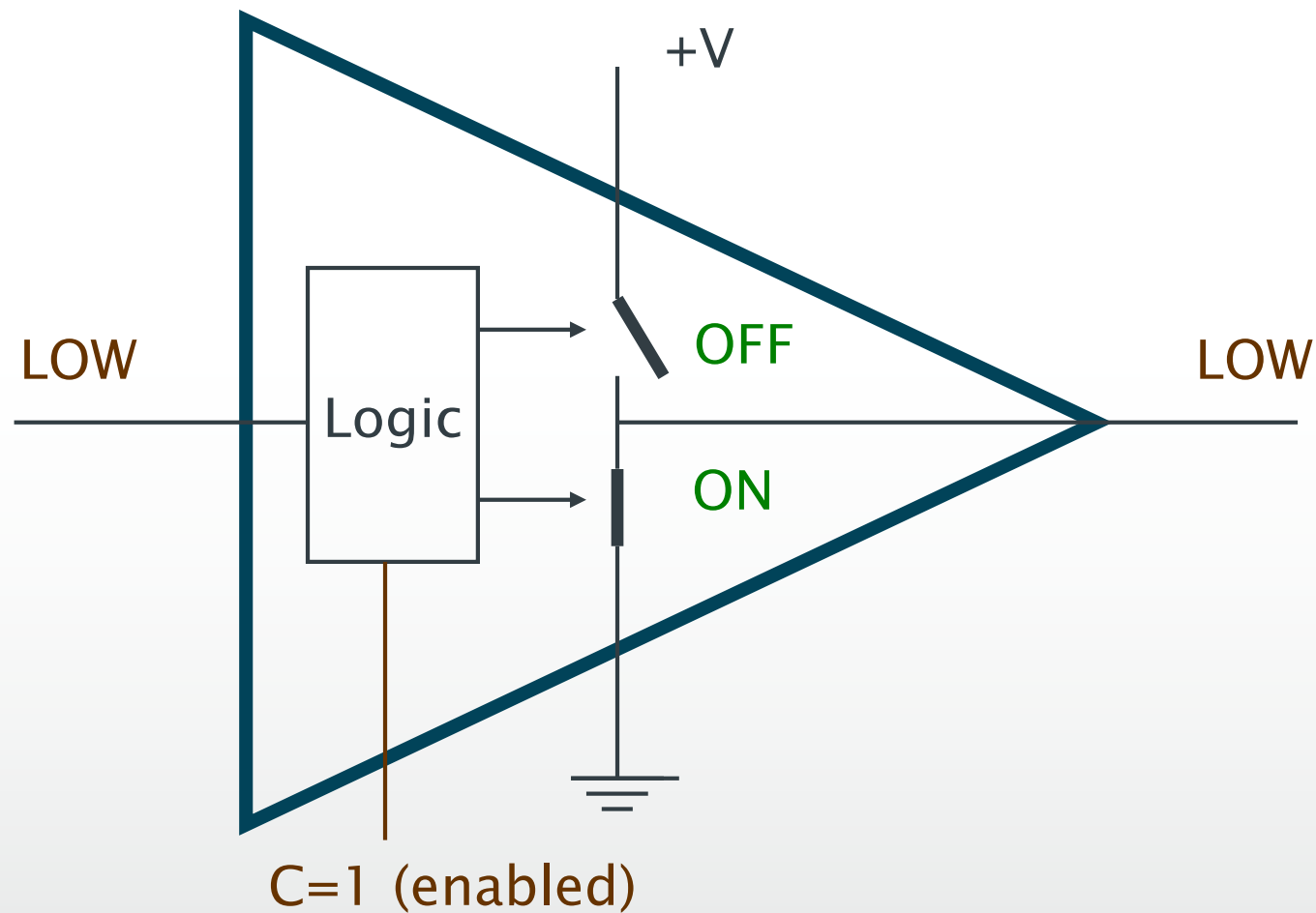
A	C	Y
0	1	0
1	1	1

The Tri-State Buffer

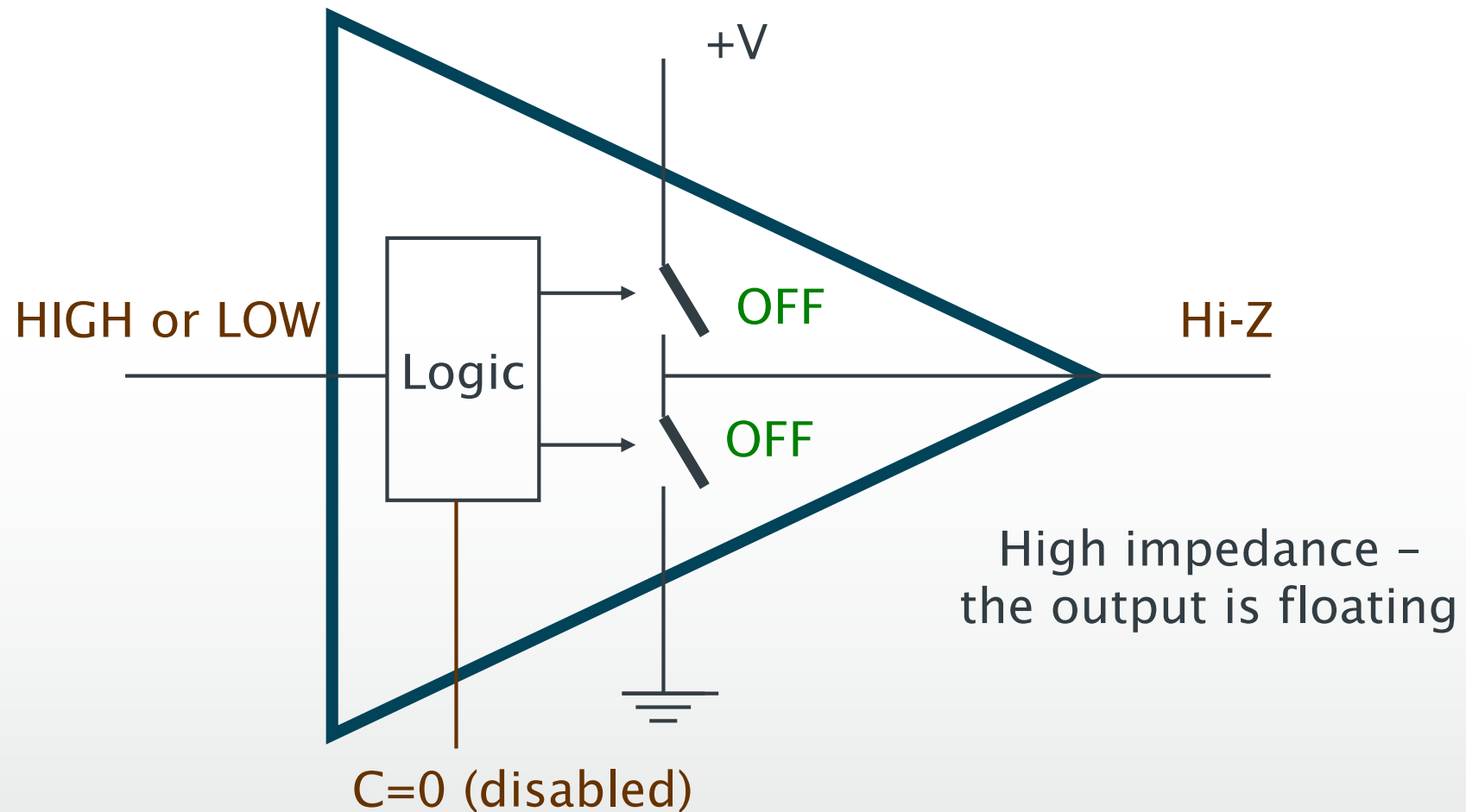
- E.g. 74HC125



The Tri-State Buffer



The Tri-State Buffer



Tristate MUX

- Connect multiple tristate buffer outputs together
- The **only** occasion when it is permissible to connect outputs of two or more gates together

Four state logic in SystemVerilog

- `bit` can take values '0' and '1'
- Three state logic needs 'Z' state (high-impedance)
- Type `logic` has four values:
 - '0' logic 0
 - '1' logic 1
 - 'Z' high impedance
 - 'X' unknown
- Note. Verilator does not support 4 state logic

logic or wire?

- logic is a "variable"
 - Has values, but no signal strength or signal resolution
- If two signals drive the same physical hardware, use "wire"
- Wires can have logic values (0, 1, Z, X) and up to 16 strengths (e.g. strong1, weak1, highz1, highz0, weak0, strong0)
- Use "continuous assignment" to drive a wire

26.2

Buses

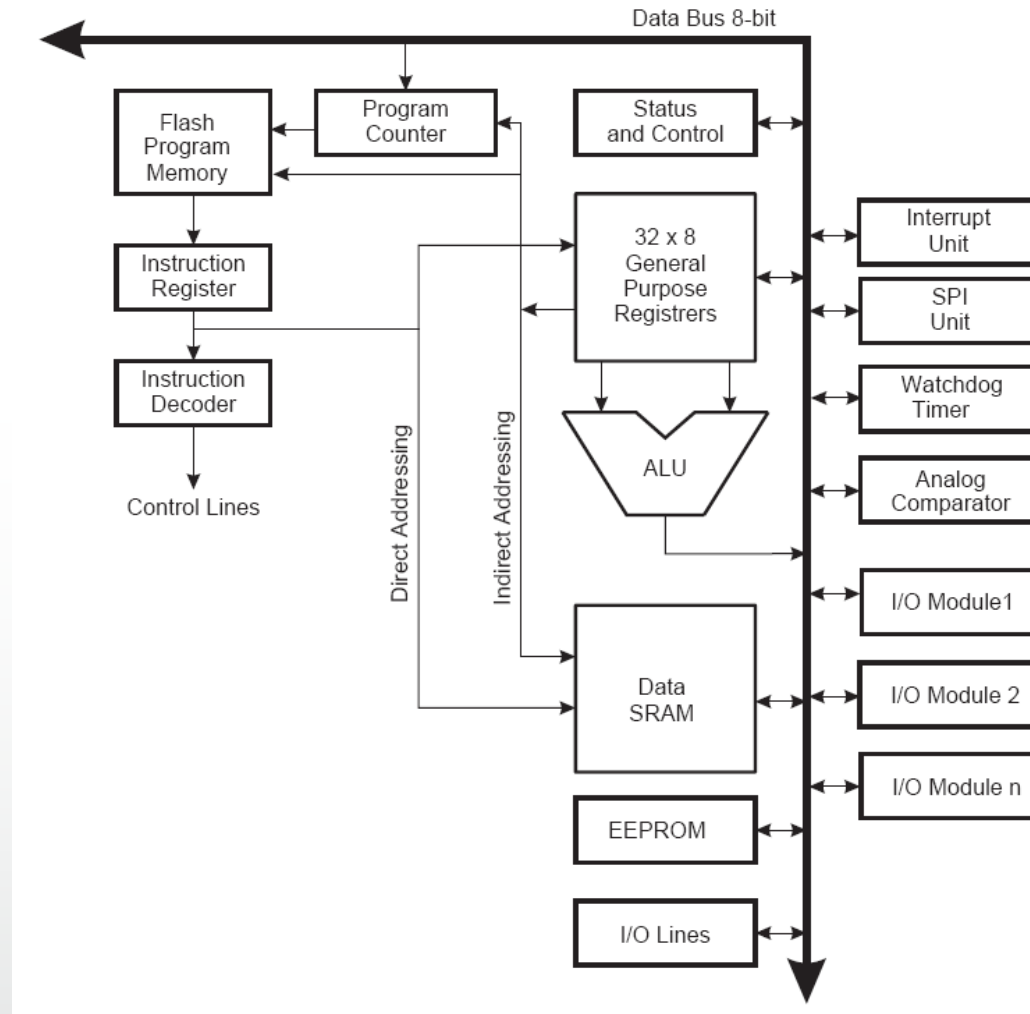
Buses

- What is a bus?

– *“A collection of two or more related signal lines”*

Wakerly, 2006

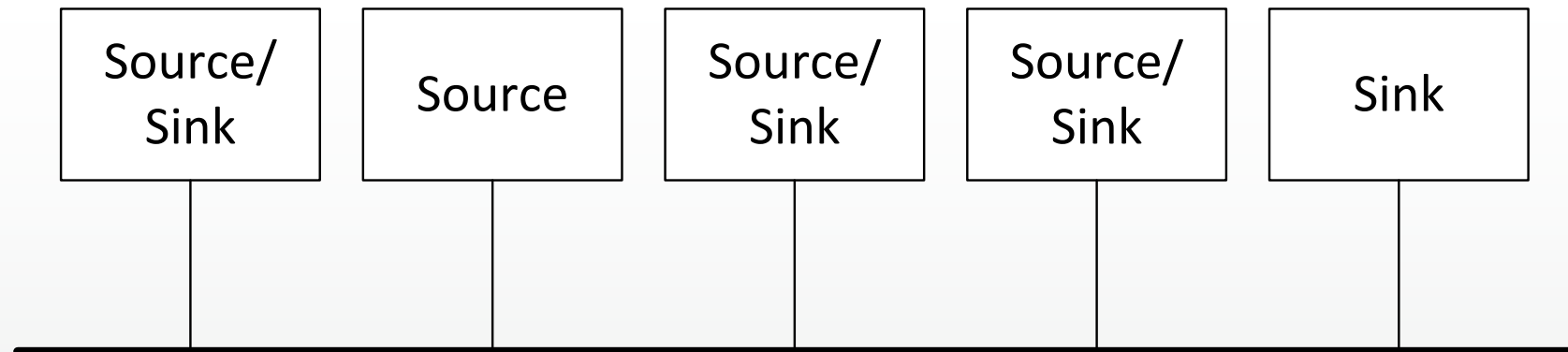
(Basic) Computer Architecture



Source: https://secure.ecs.soton.ac.uk/notes/ellabs/databook/avr/ATmega48_full.pdf

Bus Contention

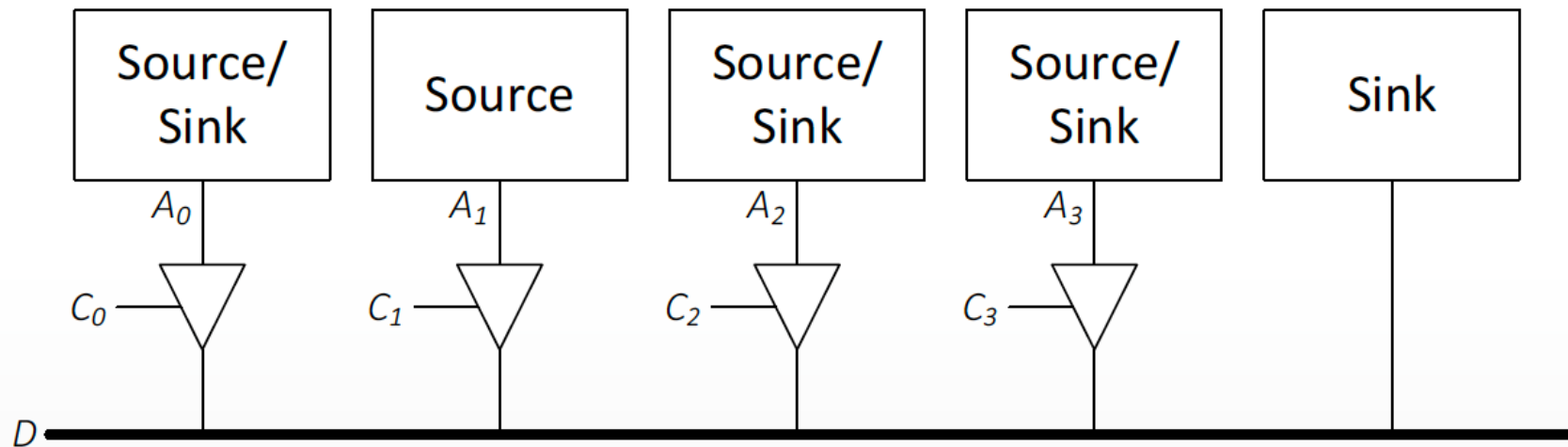
- Buses are commonly shared between multiple data sources and sinks
- This allows data to be transferred to and from them without an exponential increase in interconnect.



- Bus contention occurs when multiple signals are attempting to drive the bus at any one moment.
- This can corrupt the received signal, or potentially cause damage

Shared Buses

- We can use tri-state buffers to prevent bus contention



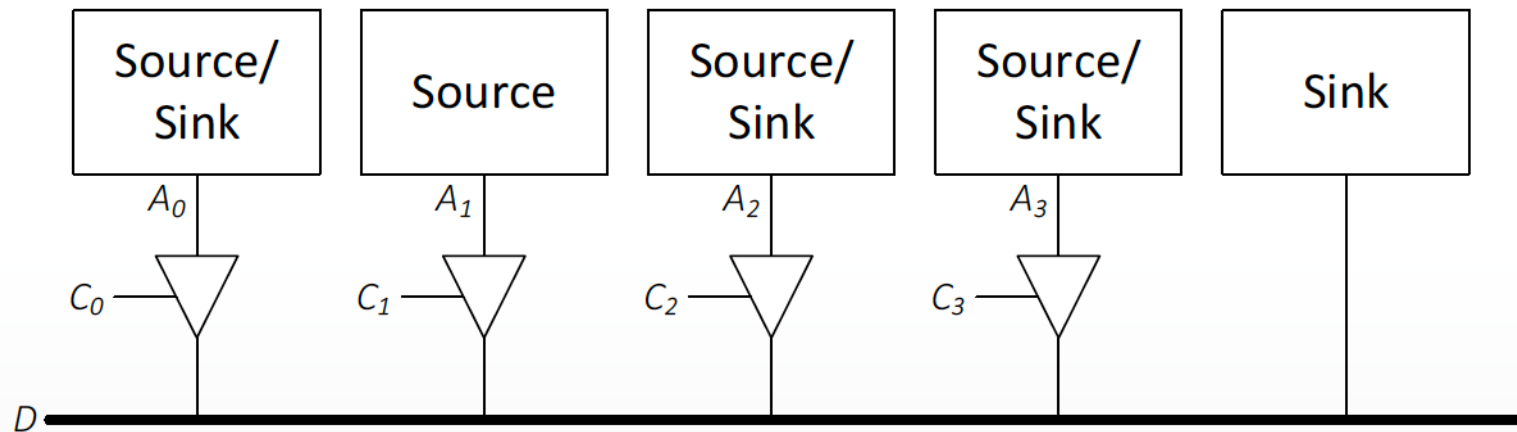
- Assume that only one, at most, of the C_x inputs is logic '1' at any time...

when C_0 then $D = A_0$
when C_1 then $D = A_1$
when C_2 then $D = A_2$

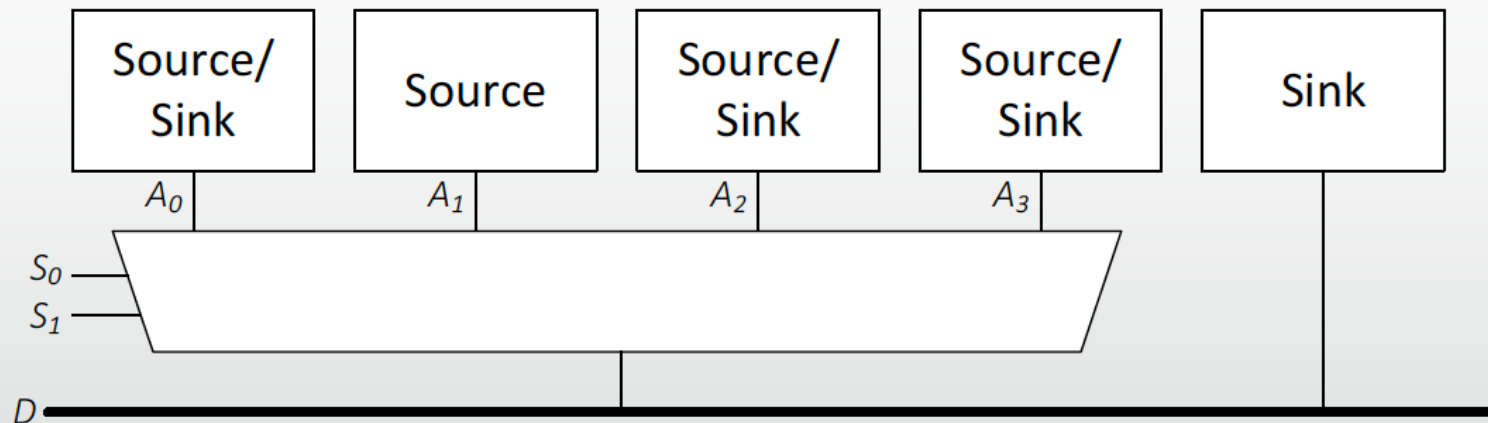
...

Buses and Multiplexers

- With tri-state buffers



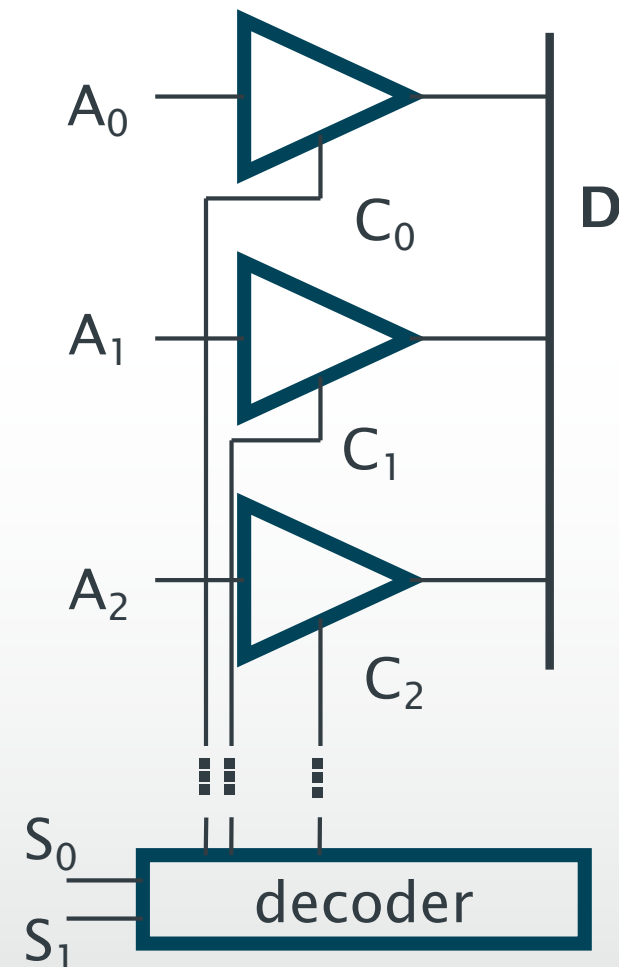
- With a multiplexer



Buses and Multiplexers

- We can therefore also use tri-states to implement the functionality of a multiplexer...

S_1	S_0	C_3	C_2	C_1	C_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0



Signal Resolution

```
module three_state_mux (output wire y,  
    input logic a0, a1, a2, a3,  
    input logic [1:0] s);
```

```
    assign y = (s == 0) ? a0 : 1'bz;
```

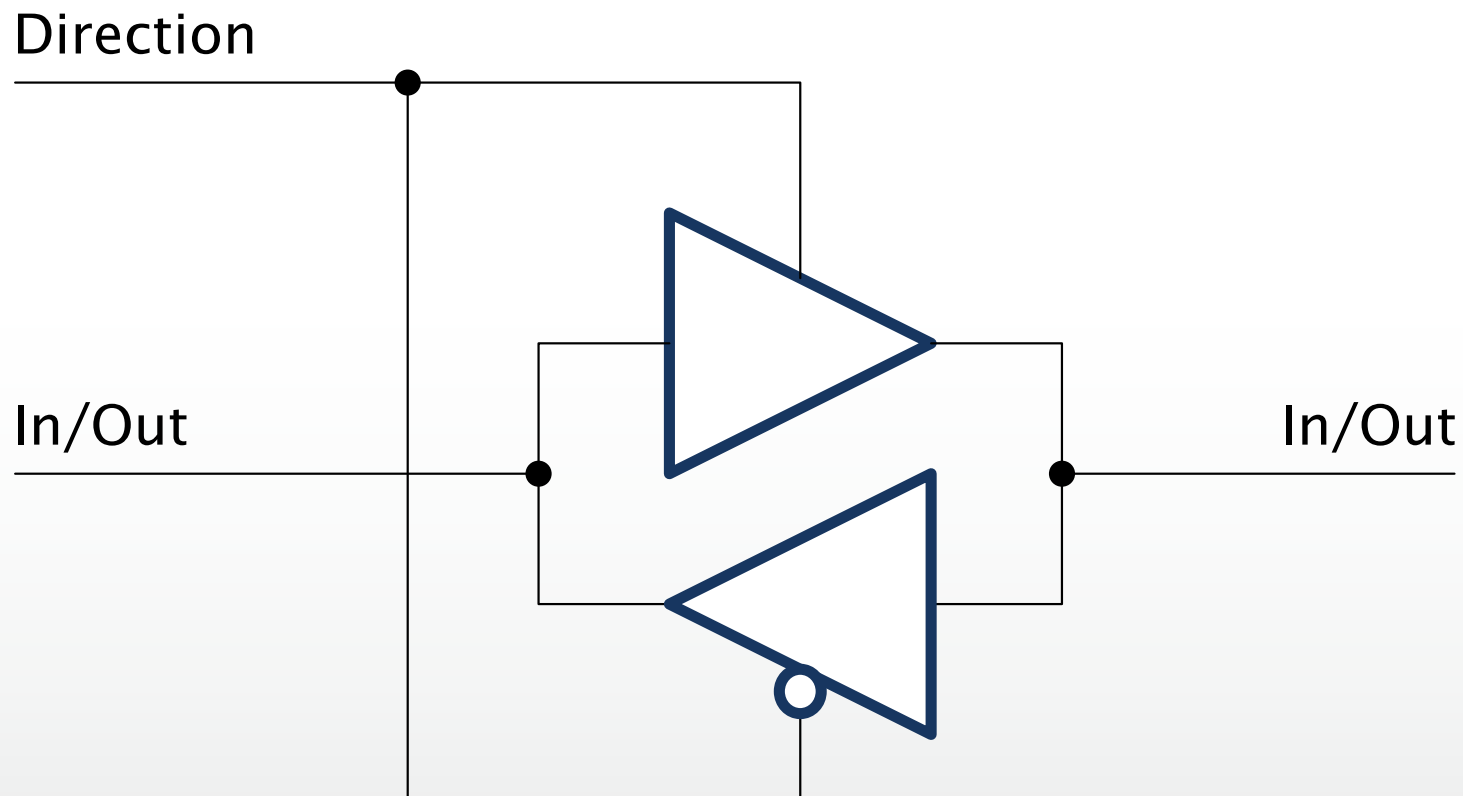
```
    assign y = (s == 1) ? a1 : 1'bz;
```

```
    assign y = (s == 2) ? a2 : 1'bz;
```

```
    assign y = (s == 3) ? a3 : 1'bz;
```

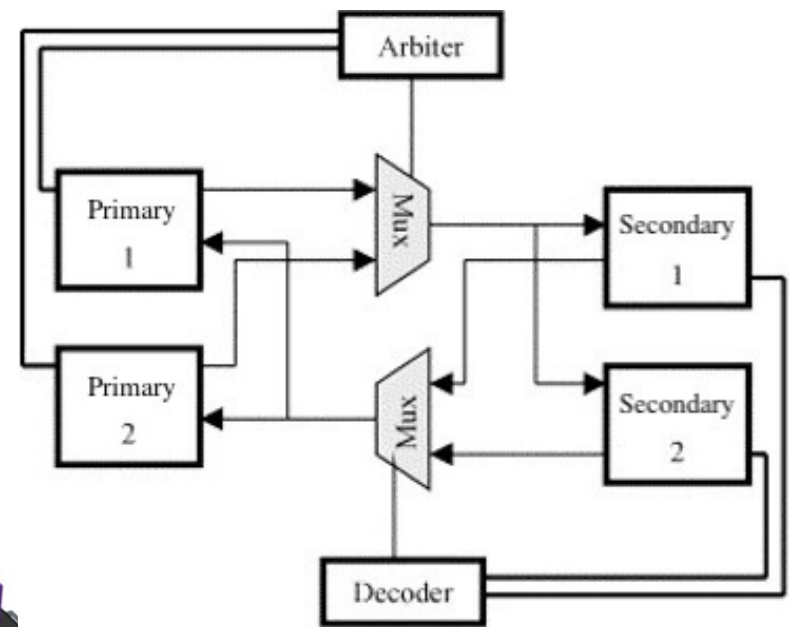
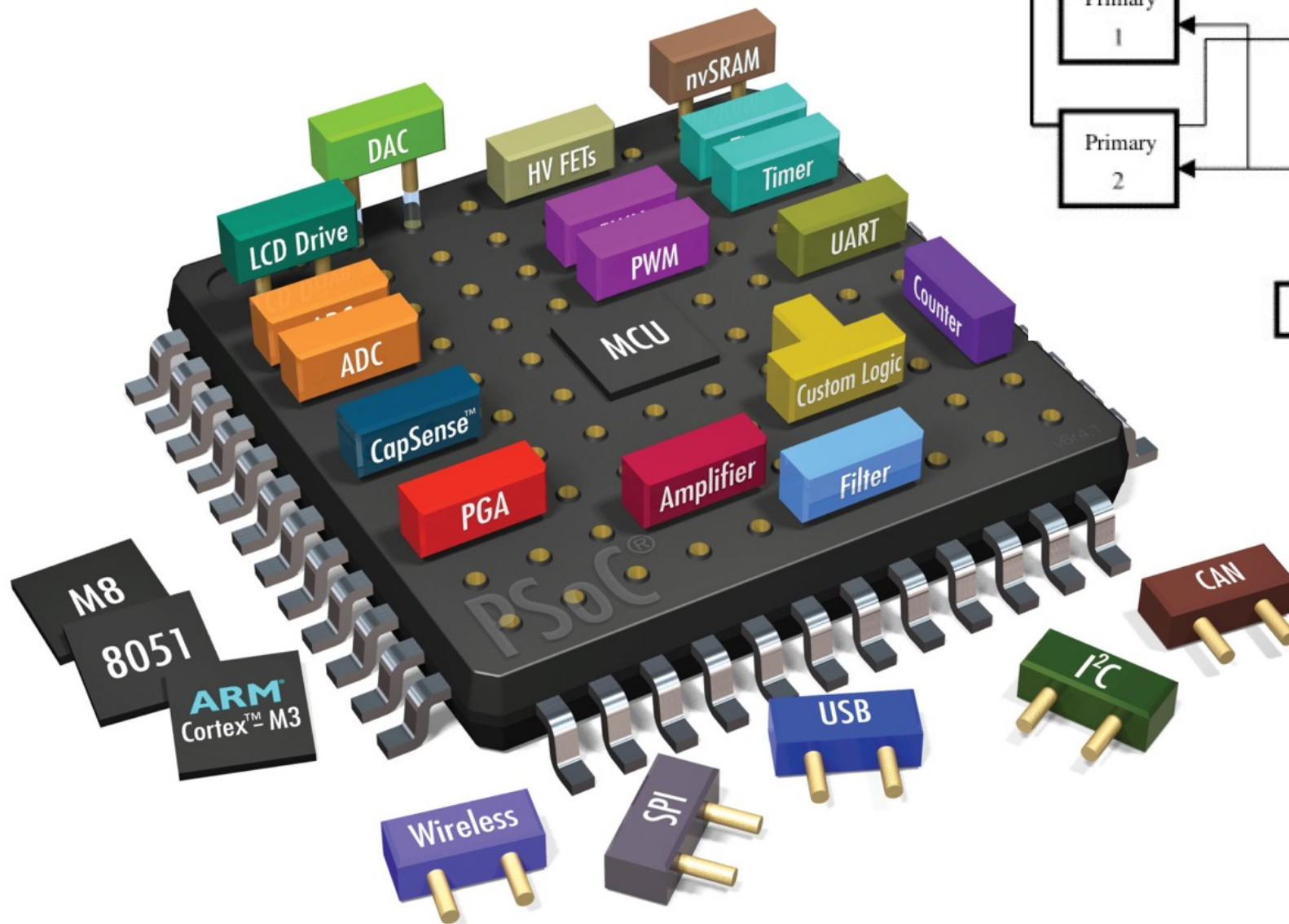
```
endmodule
```

Bidirectional Buffers



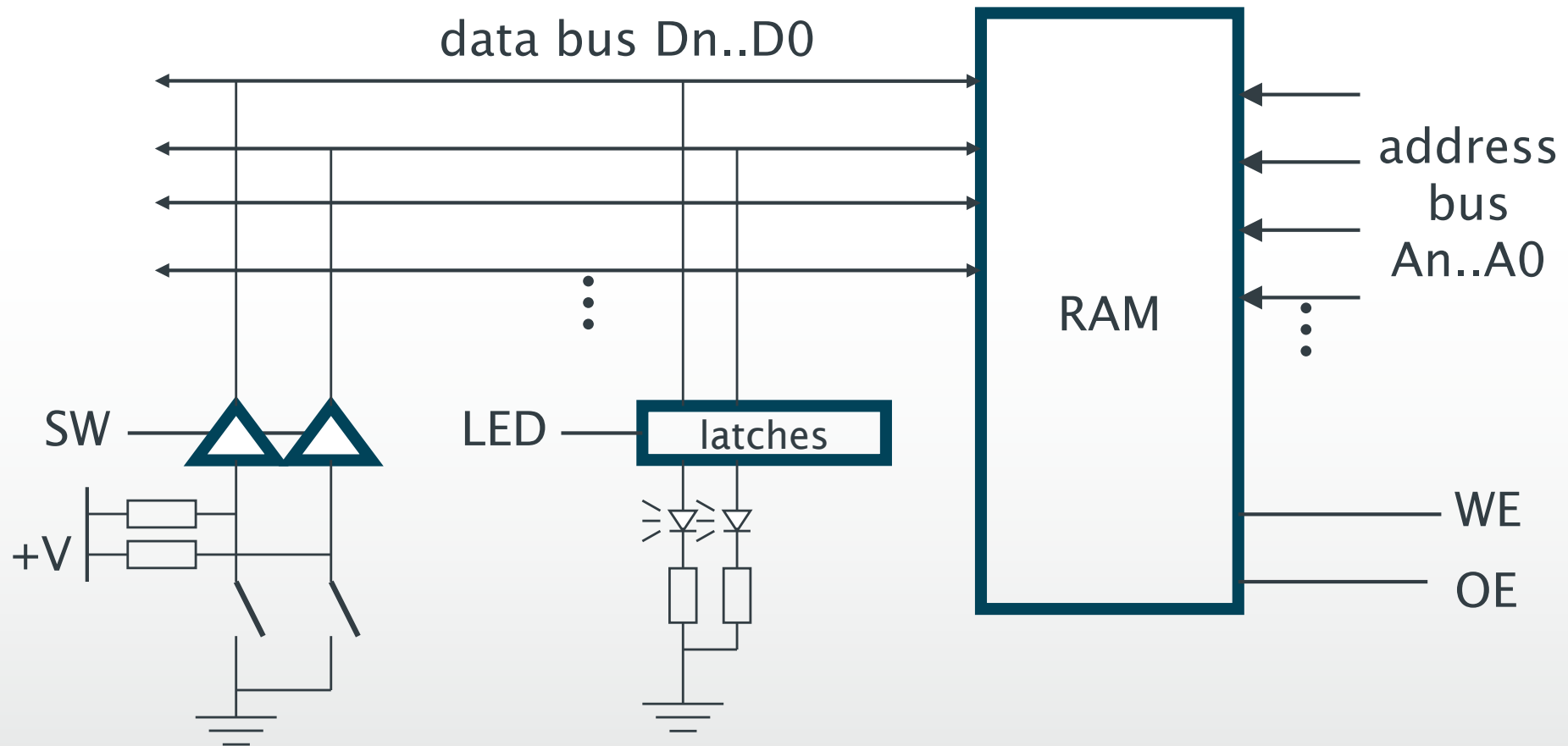
26.3

An Example



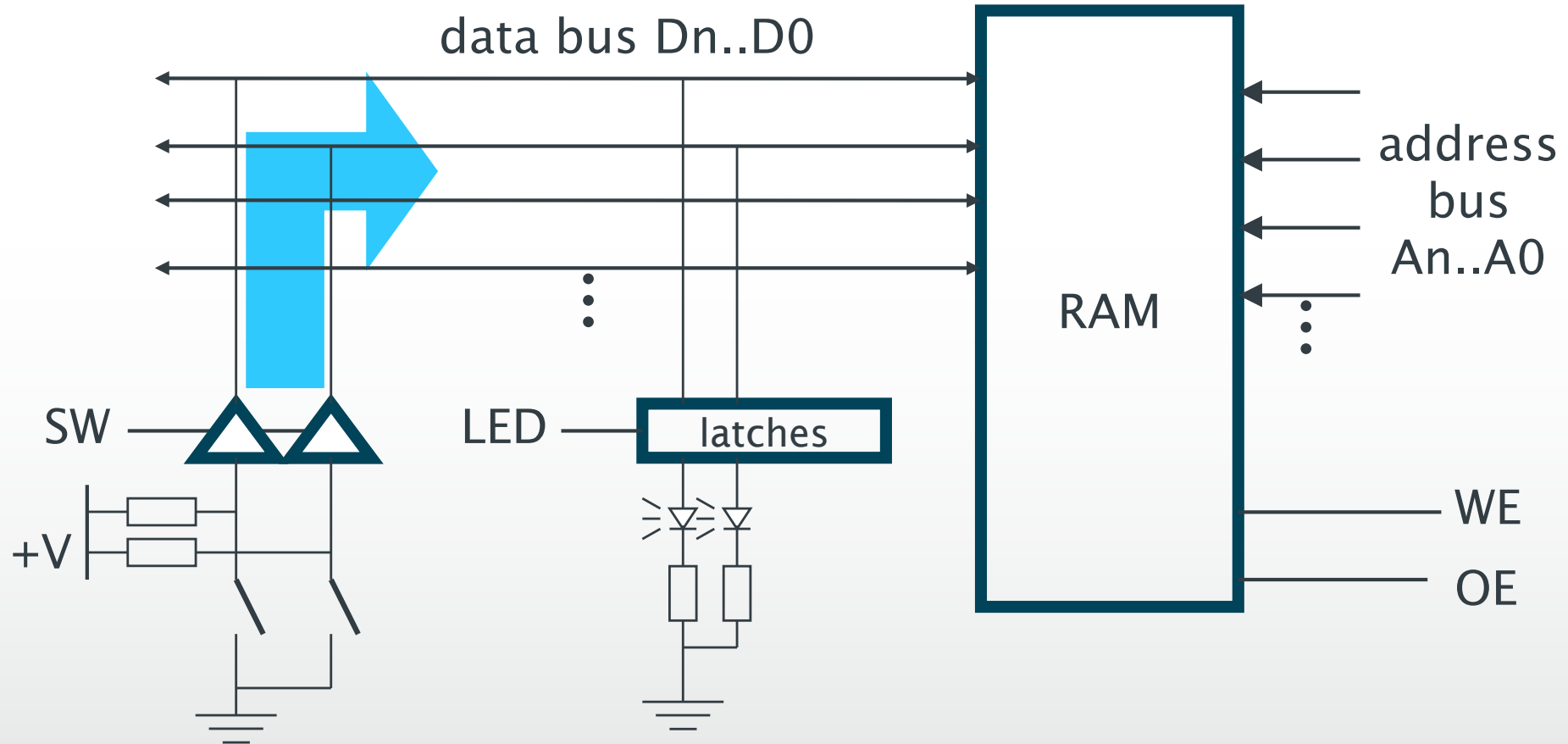
Memory Systems with Buses

- How do we copy the value of the switches onto the LEDs (going via RAM)?



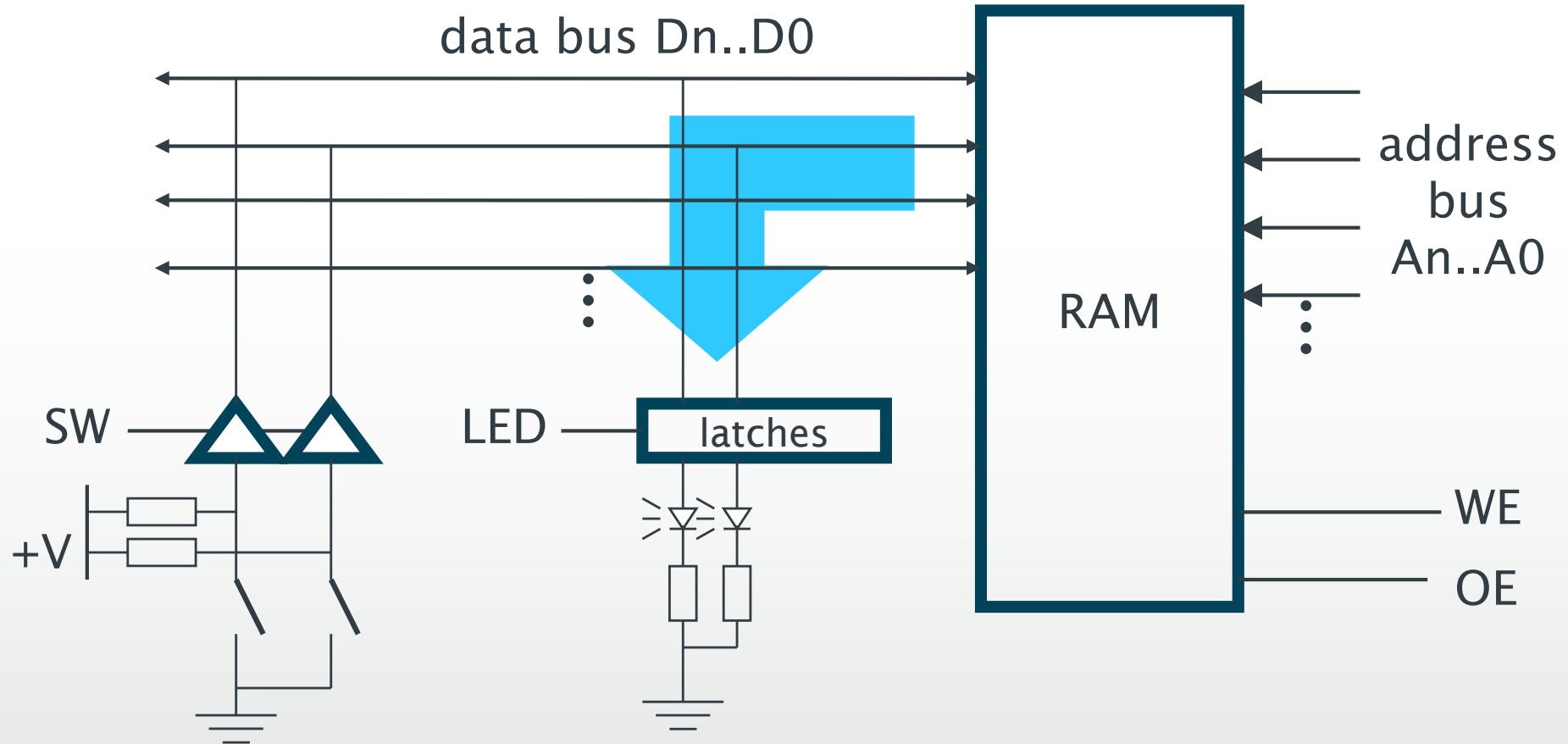
Memory Systems with Buses

A = address
SW=1, LED=0
WE=1, OE=0



Memory Systems with Buses

A = address
SW=0, LED=1
WE=0, OE=1



For you to do

- Look at the last three slides.
- What is the state of the bus if both SW and OE are both 1?
- What is the state of the bus if both SW and OE are both 0?

Summary

- Tri-state logic and tri-state buffers
- Pull-up and pull-down resistors
- Buses and bus contention
- Avoiding bus contention using tri-states/multiplexers